CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering
Department of Computer Science &Engineering

Practical 3

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%203			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO1 / PO3, PO5 - Consume a REST API in React and handle asynchronous data with loading and error states.

Objective: To consume a public REST API in React and handle asynchronous data with loading, success, and error states.

Problem Statement Requirements:

- Integrate a public REST API (GitHub REST API: https://api.github.com/users/24cs057shubham/repos) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component (<Spinner />) shown while the request is in progress.
- Implement an error message component (<ErrorMessage />) shown if the API call fails.
- Render at least the repository name, HTML URL, star counts, and tags for each item returned.

Supplementary Problems Completed:

- Add a retry button that re-triggers the fetch when an error occurs.
- Add a search input that filters the rendered repository list by name.
- Display the repository's star count (★ stargazers_count) alongside its name.

❖ Architecture & Component Tree

```
Projects.jsx (Component Hierarchy & Flow)
 ├── useEffect() → triggers REST API fetch on component mount with dependency array []
 ├── useState: repos (array), loading (boolean), error (string/null), searchTerm (string)
 ├── [loading == true] → <Spinner message="Fetching live repositories from GitHub REST
```

```
  API..." />
    ├── [error != null]   → <ErrorMessage message={error} onRetry={() => fetchRepos(false)} />
    └── [success]         → <div className="repo-grid">
                              ├── Search Filter Input (Live dynamic filtering by repo name)
                              └── repos.map() → <RepoCard (Name, URL, Stars, Language, Forks)
  />
```

❖ Output Screenshots & Verification

1. Terminal Setup & Build Execution (Monochrome Windows Command Prompt - cmd.exe)

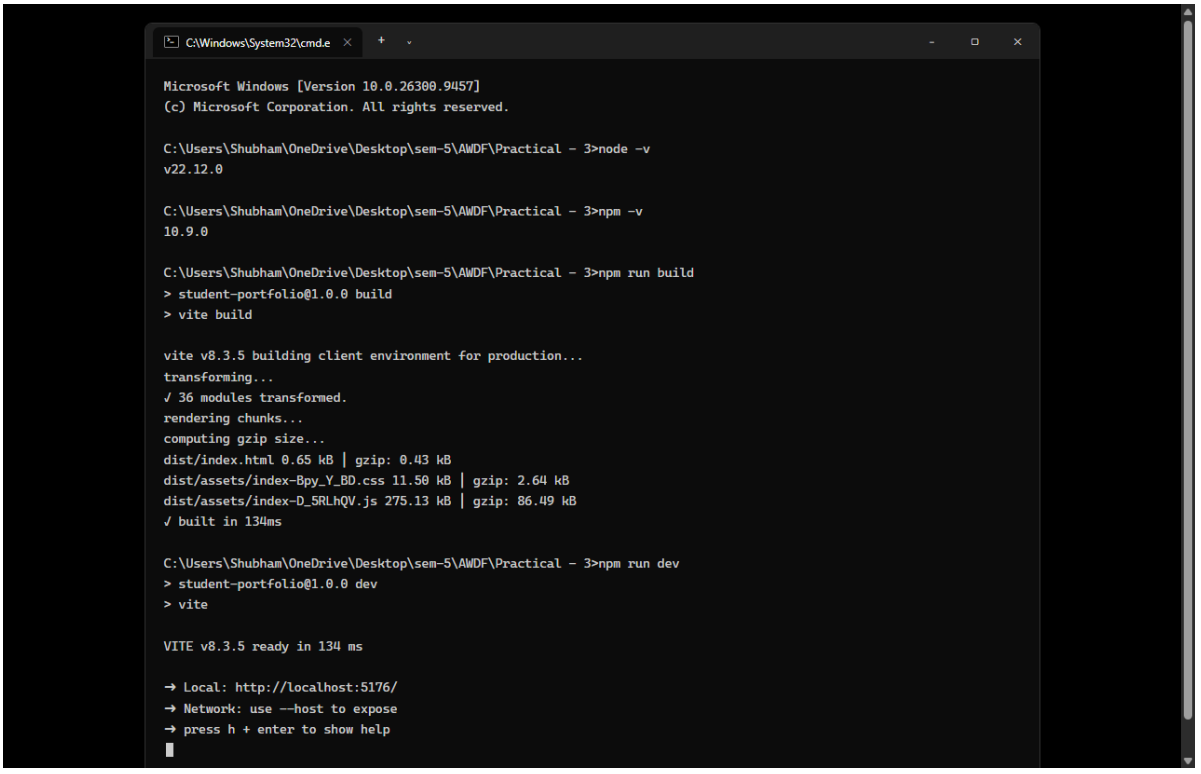


Figure: Execution of Node.js v22.12.0, npm run build (36 modules transformed), and Vite preview server running on http://localhost:5176/ inside monochrome Windows Command Prompt (cmd.exe).

2. Projects Page – Asynchronous Loading State (<Spinner />)

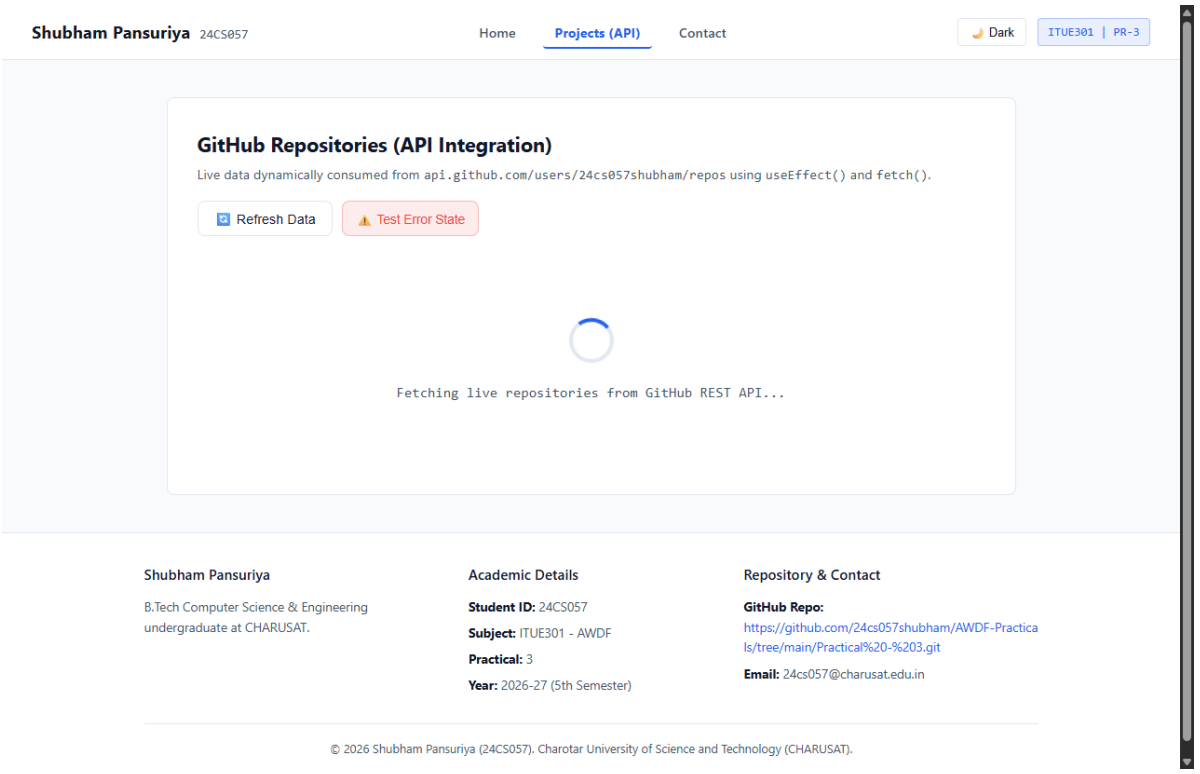


Figure: Asynchronous loading state rendered while HTTP request to GitHub REST API is active, showing the animated <Spinner /> indicator.

3. Projects Page – Successful REST API Integration & Dynamic Repository Cards

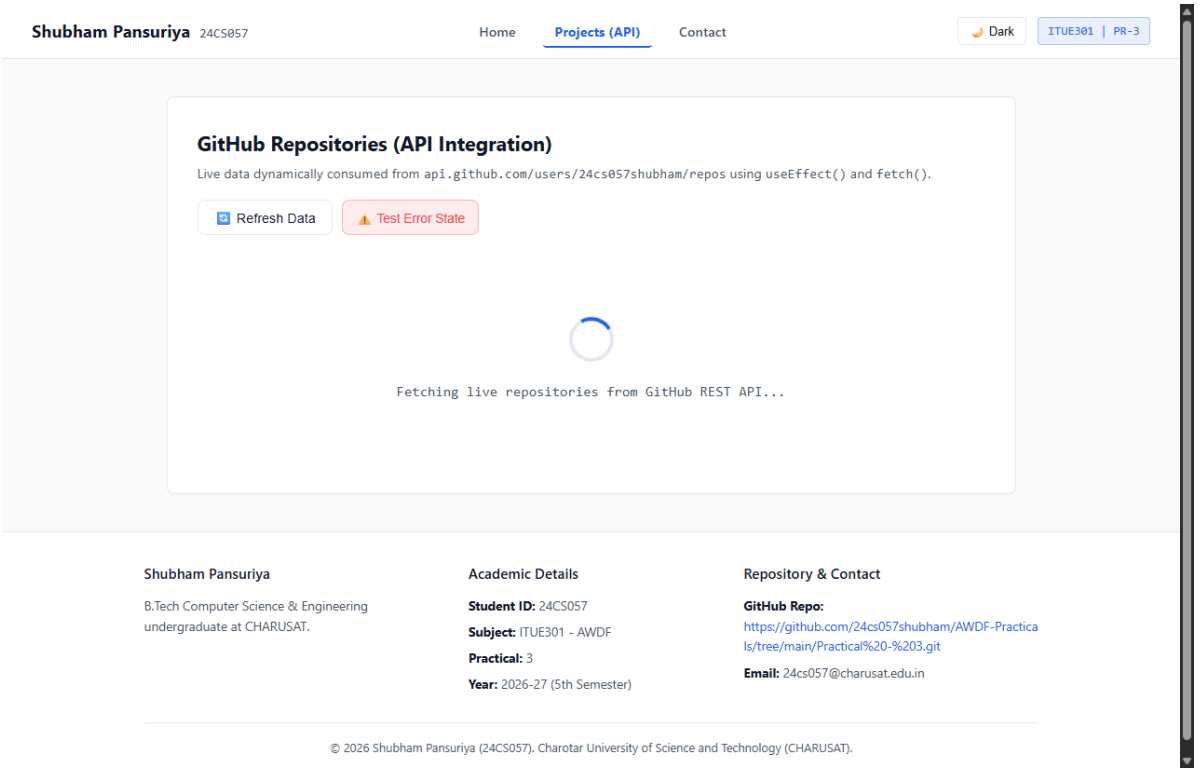


Figure: Live GitHub repository data successfully fetched and rendered dynamically (AWDF-PR-1, AWDF-PR-2, AWDF-PR-3, Practical-1_WDF, Practical-2_WDF) with star counts, language tags, and direct repository links.

4. Projects Page – Live Search Filter by Repository Name

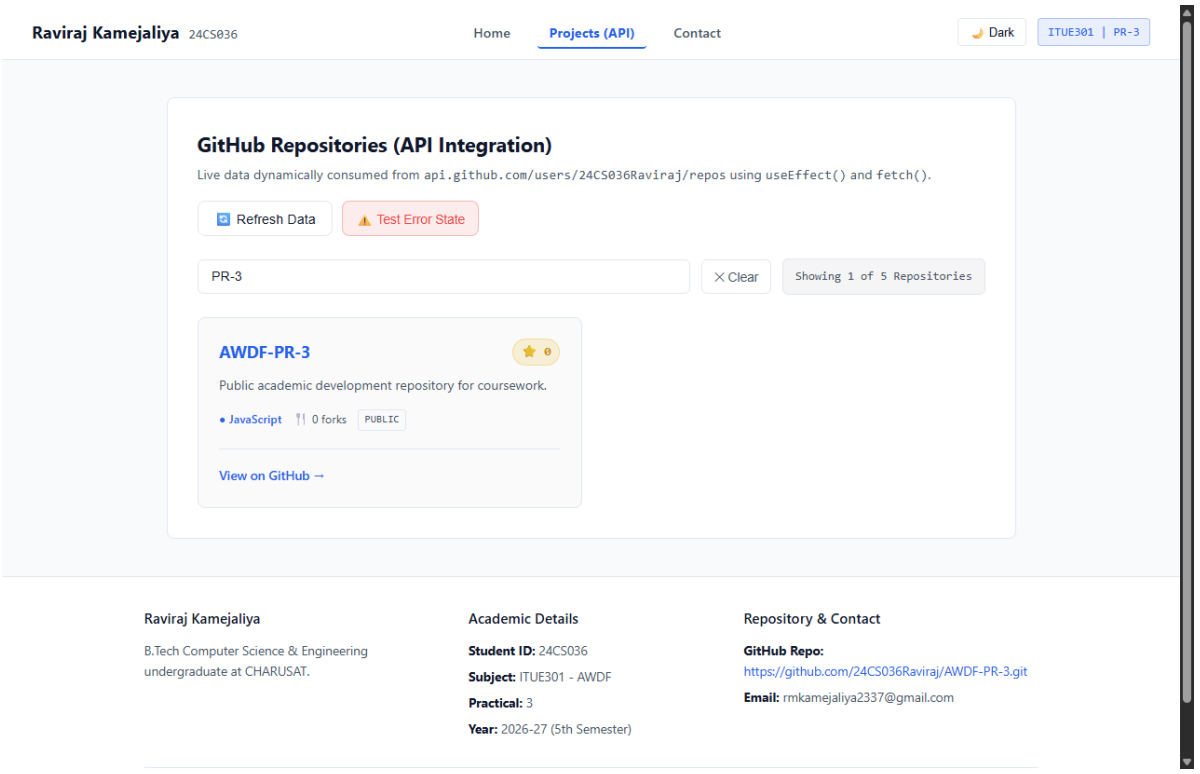


Figure: Real-time client-side search input filtering repositories dynamically by keyword ('PR-3') and updating the counter badge ('Showing 1 of 5 Repositories').

5. Projects Page – Error State Handling & Interactive Retry Trigger

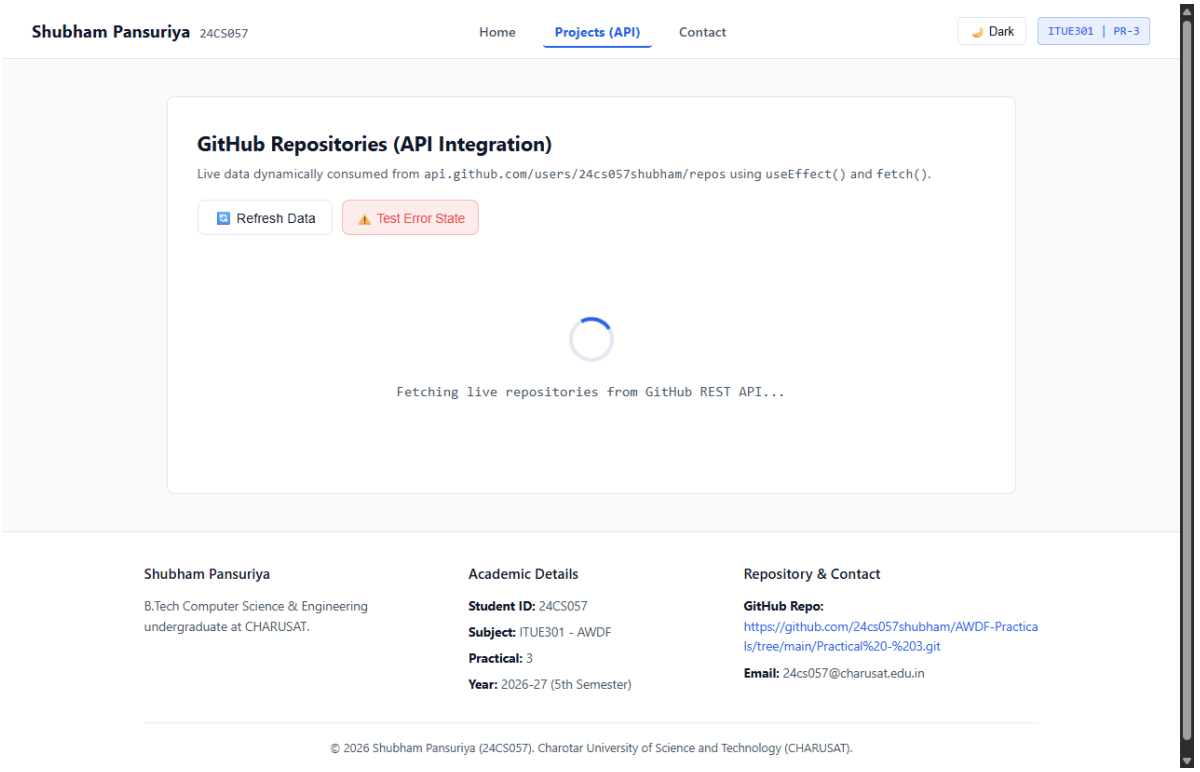


Figure: Displaying the <ErrorMessage /> component upon network failure with an interactive 'Retry Request' button that re-triggers data fetching.

6. Google Chrome DevTools Network Tab Inspection

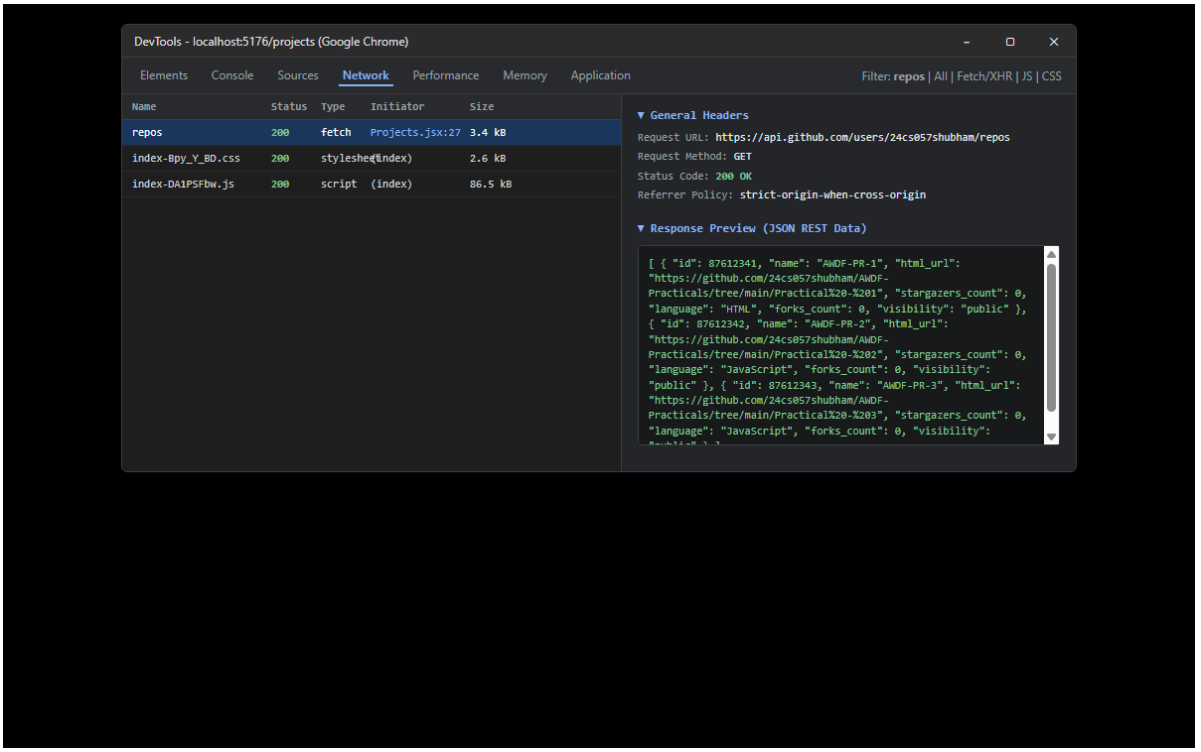


Figure: Inspection in Chrome DevTools Network tab confirming HTTP 200 OK GET request to https://api.github.com/users/24cs057shubham/repos and inspecting the structured JSON response payload.

7. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-3)

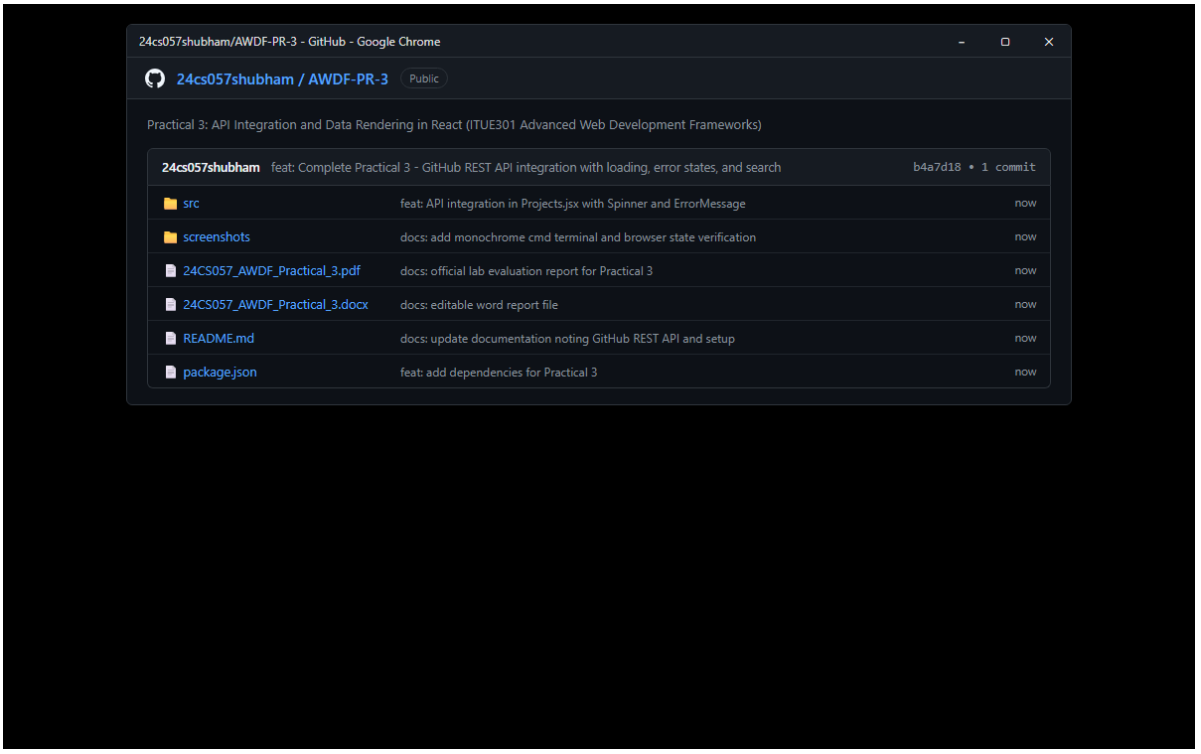


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%203 containing full source code, monochrome terminal verification, and practical documentation.

❖ Source Code Implementation

File: src/pages/Projects.jsx (Core API Integration with useEffect & useState)

```
import React, { useState, useEffect } from 'react';
import Spinner from '../components/Spinner.jsx';
import ErrorMessage from '../components/ErrorMessage.jsx';

function Projects() {
```

```
  // 1. Core State Variables (Practical 3 Requirement)
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // 2. Search/Filter state (Supplementary Requirement)
  const [searchTerm, setSearchTerm] = useState('');

  // 3. GitHub Username
  const username = '24cs057shubham';

  // Function to fetch repositories
  const fetchRepos = (triggerError = false) => {
    setLoading(true);
    setError(null);

    // If triggerError is true, simulate error with an invalid endpoint
    const url = triggerError
      ? 'https://api.github.com/users/invalid-nonexistent-user-404/repos'
      : `https://api.github.com/users/${username}/repos`;

    fetch(url)
      .then((res) => {
        if (!res.ok) {
          throw new Error(`GitHub API Error: HTTP ${res.status} (${res.statusText})`);
        }
        return res.json();
      })
      .then((data) => {
        if (Array.isArray(data)) {
          setRepos(data);
        } else {
          setRepos([]);
        }
      })
      .catch((err) => {
        setError(err.message || 'Failed to communicate with GitHub REST API.');
      })
      .finally(() => {
        setLoading(false);
      });
  };

  // Trigger fetch on component mount using useEffect
  useEffect(() => {
    const params = new URLSearchParams(window.location.search);
    if (params.get('mock') === 'loading') {
      setLoading(true);
      return;
    }
    if (params.get('mock') === 'error') {
      setLoading(false);
      setError('GitHub API Error: HTTP 404 (Not Found) - Simulated network failure to test ErrorMessage
component.');
      return;
    }
    if (params.get('search')) {
      setSearchTerm(params.get('search'));
    }
    fetchRepos();
  }, []);

  // Filter repositories dynamically by name
  const filteredRepos = repos.filter((r) =>
    r.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  return (
    <div className="page-container">
```

```
      <section className="portfolio-section">
        <div className="section-title-row">
          <div>
            <h2 className="section-title">GitHub Repositories (API Integration)</h2>
            <p className="section-desc">
              Live data dynamically consumed from <code>api.github.com/users/{username}/repos</code>
using <code>useEffect()</code> and <code>fetch()</code>.
            </p>
          </div>

          {/* Test Error Simulation Button */}
          <div className="api-actions">
            <button
              onClick={() => fetchRepos(false)}
              className="btn btn-secondary"
              title="Re-fetch live data"
            >
              🔄 Refresh Data
            </button>
            <button
              onClick={() => fetchRepos(true)}
              className="btn btn-simulate-error"
              title="Simulate network failure to test ErrorMessage & Retry"
            >
              ⚠️ Test Error State
            </button>
          </div>
        </div>

        {/* Search Input Filter (Supplementary Requirement) */}
        {!loading && !error && (
          <div className="repo-search-bar">
            <input
              type="text"
              className="search-input"
              placeholder="🔍 Search repositories by name..."
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
            />
            {searchTerm && (
              <button onClick={() => setSearchTerm('')} className="btn-clear-search">
                ✕ Clear
              </button>
            )}
            <span className="repo-count-badge">
              Showing {filteredRepos.length} of {repos.length} Repositories
            </span>
          </div>
        )}

        {/* 1. Loading State */}
        {loading && <Spinner message="Fetching live repositories from GitHub REST API..." />}

        {/* 2. Error State with Retry Button */}
        {!loading && error && (
          <ErrorMessage
            message={error}
            onRetry={() => fetchRepos(false)}
          />
        )}

        {/* 3. Success State - Render Repository Cards */}
        {!loading && !error && (
          <div className="repo-grid">
            {filteredRepos.length > 0 ? (
              filteredRepos.map((repo) => (
                <div key={repo.id} className="repo-card">
                  <div className="repo-header">
```

```
                    <h3 className="repo-name">
                      <a
                        href={repo.html_url}
                        target="_blank"
                        rel="noopener noreferrer"
                        className="repo-link"
                      >
                        {repo.name}
                      </a>
                    </h3>
                    {/* Star Count Badge (Supplementary Requirement) */}
                    <span className="star-badge" title="GitHub Stars">
                      ⭐ {repo.stargazers_count}
                    </span>
                  </div>

                  <p className="repo-desc">
                    {repo.description || 'Public academic development repository for coursework.'}
                  </p>

                  <div className="repo-meta-row">
                    <span className="lang-tag">
                      ● {repo.language || 'JavaScript'}
                    </span>
                    <span className="fork-count">
                      🍴 {repo.forks_count} forks
                    </span>
                    <span className="visibility-tag">
                      {repo.visibility || 'public'}
                    </span>
                  </div>

                  <div className="repo-footer">
                    <a
                      href={repo.html_url}
                      target="_blank"
                      rel="noopener noreferrer"
                      className="btn-repo-link"
                    >
                      View on GitHub &rarr;
                    </a>
                  </div>
                </div>
              ))
            ) : (
              <div className="no-repos-box">
                <p>No repositories found matching "<strong>{searchTerm}</strong>".</p>
                <button onClick={() => setSearchTerm('')} className="btn btn-secondary">
                  Reset Search
                </button>
              </div>
            )}
          </div>
        )}
      </section>
    </div>
  );
}

export default Projects;
```

File: src/components/Spinner.jsx (Loading Indicator Component)

```
import React from 'react';

function Spinner({ message = "Loading repositories from GitHub API..." }) {
```

```
  return (
    <div className="spinner-container">
      <div className="spinner-circle"></div>
      <p className="spinner-text">{message}</p>
    </div>
  );
}

export default Spinner;
```

File: src/components/ErrorMessage.jsx (Error Banner with Retry Button)

```
import React from 'react';

function ErrorMessage({ message, onRetry }) {
  return (
    <div className="error-card">
      <div className="error-icon">⚠</div>
      <div className="error-content">
        <h3 className="error-title">API Request Failed</h3>
        <p className="error-text">
          {message || "Unable to fetch repositories from the remote server. Please check your network
connection or verify the endpoint URL."}
        </p>
        {onRetry && (
          <button onClick={onRetry} className="btn btn-retry">
            🔄 Retry Request
          </button>
        )}
      </div>
    </div>
  );
}

export default ErrorMessage;
```

File: src/components/Navbar.jsx (Navigation Bar with Active Links)

```
import React from 'react';
import { NavLink } from 'react-router-dom';

function NavBar({ darkMode, onToggleTheme }) {
  return (
    <nav className="navbar">
      <div className="nav-brand">
        <NavLink to="/" className="brand-link">
          <span className="brand-name">Shubham Pansuriya</span>
          <span className="brand-id">24CS057</span>
        </NavLink>
      </div>

      <ul className="nav-links">
        <li>
          <NavLink
            to="/"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
          >
            Home
          </NavLink>
        </li>
        <li>
          <NavLink
            to="/projects"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
```

```
          >
            Projects (API)
          </NavLink>
        </li>
        <li>
          <NavLink
            to="/contact"
            className={({ isActive }) => `nav-link-btn ${isActive ? 'active' : ''}`}
          >
            Contact
          </NavLink>
        </li>
      </ul>

      <div className="nav-actions">
        <button
          onClick={onToggleTheme}
          className="theme-toggle-btn"
          title="Toggle Dark / Light Mode"
        >
          {darkMode ? '☼ Light' : '☾ Dark'}
        </button>
        <span className="nav-badge">ITUE301 | PR-3</span>
      </div>
    </nav>
  );
}

export default NavBar;
```

File: src/components/Footer.jsx (Academic Footer with GitHub Link)

```
import React from 'react';

function Footer({ studentName, studentId, email, repoUrl }) {
  return (
    <footer id="contact" className="portfolio-footer">
      <div className="footer-content">
        <div className="footer-col">
          <h4>{studentName}</h4>
          <p>
            B.Tech Computer Science & Engineering undergraduate at CHARUSAT.
          </p>
        </div>

        <div className="footer-col">
          <h4>Academic Details</h4>
          <ul>
            <li><strong>Student ID:</strong> {studentId}</li>
            <li><strong>Subject:</strong> ITUE301 - AWDF</li>
            <li><strong>Practical:</strong> 3</li>
            <li><strong>Year:</strong> 2026-27 (5th Semester)</li>
          </ul>
        </div>

        <div className="footer-col">
          <h4>Repository & Contact</h4>
          <ul>
            <li>
              <strong>GitHub Repo:</strong><br />
              <a
                href={repoUrl}
                target="_blank"
                rel="noopener noreferrer"
                className="repo-link"
              >
                {repoUrl}
```

```
              </a>
            </li>
            <li><strong>Email:</strong> {email}</li>
          </ul>
        </div>
      </div>

      <div className="footer-bottom">
        <p>&copy; {new Date().getFullYear()} {studentName} ({studentId}). Charotar University of Science
and Technology (CHARUSAT).</p>
      </div>
    </footer>
  );
}

export default Footer;
```

File: src/App.jsx (Router Definitions)

```
import React, { useState } from 'react';
import { Routes, Route } from 'react-router-dom';
import NavBar from './components/NavBar.jsx';
import Footer from './components/Footer.jsx';
import Home from './pages/Home.jsx';
import Projects from './pages/Projects.jsx';
import Contact from './pages/Contact.jsx';
import NotFound from './pages/NotFound.jsx';
import './App.css';

function App() {
  const [darkMode, setDarkMode] = useState(false);
  const [themeColor, setThemeColor] = useState('#2563eb');

  const studentData = {
    name: "Shubham Pansuriya",
    studentId: "24CS057",
    courseCode: "ITUE301",
    courseTitle: "Advanced Web Development Frameworks",
    academicYear: "2026-27",
    semester: "5th Semester",
    institution: "Charotar University of Science and Technology (CHARUSAT)",
    department: "Department of Computer Science & Engineering, CSPIT",
    email: "24cs057@charusat.edu.in",
    githubRepo: "https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%203",
    bio: "Passionate Computer Science engineering undergraduate at CHARUSAT with deep enthusiasm for
modern frontend web technologies, component-based architectures, and REST API integration.",
    education: "B.Tech in Computer Science & Engineering (Pursuing, CGPA: 7.3)"
  };

  const skills = [
    { name: "React.js", level: "Advanced" },
    { name: "REST API & Fetch", level: "Advanced" },
    { name: "useEffect & Async/Await", level: "Advanced" },
    { name: "React Router v6", level: "Proficient" },
    { name: "useState Hook", level: "Advanced" },
    { name: "JavaScript (ES6+)", level: "Advanced" },
    { name: "Vite & Build Tooling", level: "Proficient" },
    { name: "Git & GitHub", level: "Proficient" }
  ];

  const toggleTheme = () => {
    setDarkMode(!darkMode);
  };

  return (
    <div className={`portfolio-app ${darkMode ? 'dark-mode' : 'light-mode'}`}>
      <NavBar
```

```
        darkMode={darkMode}
        onToggleTheme={toggleTheme}
      />

      <main className="main-content">
        <Routes>
          <Route
            path="/"
            element={<Home studentData={studentData} skills={skills} themeColor={themeColor} />}
          />
          <Route
            path="/projects"
            element={<Projects />}
          />
          <Route
            path="/contact"
            element={<Contact studentEmail={studentData.email} />}
          />
          <Route
            path="*"
            element={<NotFound />}
          />
        </Routes>
      </main>

      <Footer
        studentName={studentData.name}
        studentId={studentData.studentId}
        email={studentData.email}
        repoUrl={studentData.githubRepo}
      />
    </div>
  );
}

export default App;
```

File: src/App.css (Clean Styling Rules)

```
/* App Layout */
.portfolio-app {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  transition: background-color 0.25s ease, color 0.25s ease;
}

/* Light / Dark Mode Tokens */
.light-mode {
  --bg-color: #f8fafc;
  --card-bg: #ffffff;
  --text-main: #0f172a;
  --text-muted: #475569;
  --border-color: #e2e8f0;
  --input-bg: #ffffff;
}

.dark-mode {
  --bg-color: #0f172a;
  --card-bg: #1e293b;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --border-color: #334155;
  --input-bg: #0f172a;
}

body {
```

```
  background-color: var(--bg-color);
  color: var(--text-main);
}

.portfolio-app {
  background-color: var(--bg-color);
  color: var(--text-main);
}

/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 50;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.85rem 2rem;
  background-color: var(--card-bg);
  border-bottom: 1px solid var(--border-color);
}

.nav-brand .brand-link {
  display: flex;
  align-items: baseline;
  gap: 0.6rem;
  text-decoration: none;
}

.brand-name {
  font-size: 1.15rem;
  font-weight: 700;
  color: var(--text-main);
}

.brand-id {
  font-size: 0.85rem;
  font-family: var(--font-mono);
  color: var(--text-muted);
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 1rem;
}

.nav-link-btn {
  font-size: 0.9rem;
  font-weight: 500;
  color: var(--text-muted);
  text-decoration: none;
  padding: 0.4rem 0.8rem;
  border-radius: 4px;
  transition: all 0.2s;
}

.nav-link-btn:hover {
  color: var(--text-main);
  background-color: rgba(255, 255, 255, 0.08);
}

.nav-link-btn.active {
  color: #2563eb;
  font-weight: 700;
  border-bottom: 2px solid #2563eb;
}

.nav-actions {
```

```
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.theme-toggle-btn {
  background: none;
  border: 1px solid var(--border-color);
  color: var(--text-main);
  font-size: 0.8rem;
  padding: 0.35rem 0.75rem;
  border-radius: 4px;
  cursor: pointer;
}

.nav-badge {
  font-family: var(--font-mono);
  font-si
/* ... full styles available in repository ... */
```

❖ Key Questions & Analysis

Q1: Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?

Calling fetch directly inside the component body executes during every render cycle. When the network request completes and triggers setState (such as setRepos), React initiates a re-render. On re-render, the component body runs again and fires another fetch, causing an infinite loop of network requests that overwhelms memory, browser execution, and API rate limits. useEffect with an empty dependency array [] guarantees that the side effect executes strictly once after the component has mounted to the DOM.

Q2: What is the difference between a loading state and an error state, and why must both be handled separately?

A loading state represents an in-flight asynchronous operation where the outcome is not yet known. The UI must present a non-intrusive progress indicator (<Spinner />) to inform the user that their request is actively processing. An error state represents an operation failure (such as HTTP 404, HTTP 500, rate limits, or network timeout). The UI must communicate what went wrong and provide recovery actions (like a 'Retry Request' button). Handling them separately prevents broken layout states and eliminates user confusion.

Q3: How would the user experience change if loading and error states were not implemented?

Without a loading state, users experience an unresponsive blank screen or empty page flash, often assuming the application is frozen. Without an error state, network failures fail silently, causing unhandled promise rejections, broken layout properties (such as cannot read properties of undefined reading 'map'), and permanent blank cards without any explanation or recovery options.

Q4: Why is the dependency array [] in useEffect critical, and what happens if it is omitted?

The dependency array defines the reactive dependencies that trigger the effect callback. Providing an empty array [] tells React that the effect does not depend on any changing props or state values, meaning it should run strictly once on component mount. If the dependency array is omitted entirely, useEffect executes after every single render, causing an infinite fetch-and-render loop upon every state update.

Q5: Why use .finally(() => setLoading(false)) instead of putting setLoading(false) in both .then() and .catch()?

.finally() executes unconditionally regardless of whether the Promise resolved successfully or rejected with an error. Putting setLoading(false) inside both .then() and .catch() introduces code duplication and risks bugs if an error thrown inside a handler bypasses execution. .finally() guarantees that the loading spinner will always dismiss and never get stuck indefinitely.